

Project Report: **Houseplant Watering** **Scheduler**

Faculty: Shamim Al Mamun (SAM3)

Department: CSE

Course : CSE215

Group Members:

<u>Name</u>	<u>ID</u>
Adiyat Mahmud Neel	2534687642
Ashan Moni	2532539042
Md Junayed Hossain	2533332642

Submission date:

1) Introduction

The Houseplant Watering Scheduler is a Java-based desktop application designed to solve a common household problem: remembering when to water different types of indoor plants. By utilizing a graphical user interface (GUI) built with Java Swing and a file-based storage system, the application allows users to add plants, view their status in real-time, and receive visual alerts when a plant requires watering. The architecture is heavily driven by core Object-Oriented Programming (OOP) principles.

2) Objectives

The primary goals of this project were to:

- Provide a functional tool to track watering schedules for various plant types.
- Demonstrate **Abstraction & Inheritance** by creating an abstract base class (**Plant**) with specialized subclasses (**Succulent**, **Fern**) to manage common states and specific scheduling logic.
- Implement **Polymorphism** by overriding a core status-checking method (**checkNeedsWatering()**) to handle different hardcoded moisture requirements.
- Enforce **Encapsulation** by hiding the "last watered date" variable.
- Establish an **Interface** (**Alertable**) that triggers a visual warning when a plant is overdue.
- Develop a responsive **GUI Infrastructure** using Java Swing and JPanel grid where each plant is a card that turns red (via **setBackground**) when thirsty.

3) Outcomes

The implemented code successfully fulfills the project objectives through the following technical achievements:

- **Core OOP Structure:** The application utilizes an abstract `Plant` class that defines common properties like name and last watered date. The `Succulent` and `Fern` subclasses successfully inherit these traits while defining their own interval rules (e.g., 7 days for Succulents, 15 seconds for Ferns for the sake of demonstration of a working product).
- **Polymorphism:** The `checkNeedsWatering()` method evaluates time differentials using `java.time.LocalDateTime` and behaves polymorphically depending on the specific plant instance being evaluated.
- **Encapsulated State:** Direct access to variables like `lastWateredDate` is restricted/hidden (marked as `private`), and updates are handled securely via setter methods or watering action listeners.
- **Functional GUI:** The interface uses a `JFrame` with a `GridLayout` to display individual `PlantCard` panels. The application features a live timer that automatically transitions plant cards from a healthy green background to a thirsty red background based on real-time interval calculations.
- **Interface Implementation:** The `Alertable` interface guarantees that all plants have a `triggerWarning()` mechanism. The logic was successfully integrated with a state flag to ensure visual pop-ups alert the user once per cycle without freezing the application loop.
- **Data Persistence:** The software reliably reads from and writes to a `plants.txt` file, ensuring user data is saved across sessions.

4) Conclusion

The Houseplant Watering Scheduler is a structurally sound application that successfully maps real-world requirements to fundamental Object-Oriented Programming principles. The code cleanly separates the underlying data from the Swing-based visual layer. By adhering to interface, inheritance, abstraction and encapsulation, the software remains potentially highly extensible, allowing for new plant types or alternate UI views to be integrated in the future with minimal refactoring.

5) References

- **Requirement Specifications:** Project criteria provided by professor Shamim Al Mamun.